KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

HOTEL & RESTAURANT BILLING SYSTEM

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
ROHITH S K	25BCS141	CS - 2025
ROSHINI KUMARI P	25BCS142	CS - 2025
SACHIN K	25BCS143	CS - 2025
SANJAY D	25BCS144	CS - 2025
SANTHANALAKSHMI N	25BCS145	CS - 2025

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled "**HOTEL & RESTAURANT BILLING SYSTEM"** submitted in partial fulfilment of the requirements for the course [**Course Name]** is a **record of our original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
ROHITH S K	25BCS141	CS - 2025	
ROSHINI KUMARI P	25BCS142	CS - 2025	
SACHIN K	25BCS143	CS - 2025	
SANJAY D	25BCS144	CS - 2025	
SANTHANALAKSHMI N	25BCS145	CS - 2025	

Place: COIMBATORE

Date: 16 . 03. 2026

CERTIFICATE

This is to certify that the project entitled **"HOTEL & RESTAURANT BILLING SYSTEM"** is a **bona fide work** carried out by the following students of **[Class/Batch]**, [**Department Name]**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **[Month, Year]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	ROHITH S K	25BCS141
2.	ROSHINI KUMARI P	25BCS142
3.	SACHIN K	25BCS143
4.	SANJAY D	25BCS144
5.	SANTHANALAKSHMI N	25BCS145

ABSTRACT

The project **"Hotel & Restaurant Billing System"** is developed in the **C programming language** to automate the billing operations of a restaurant. The main objective of this project is to simplify and streamline the process of order management, calculation of bills, and record keeping within a hotel or restaurant environment.

The system displays a predefined **menu list** with corresponding prices and allows users to enter multiple orders by selecting the item number and quantity. It then automatically calculates the **subtotal, applicable taxes (GST)**, and **total**. A formatted bill is generated and displayed to the customer, which can also be saved to a text file for record maintenance.

This project demonstrates the effective use of **arrays, structures, functions, and file handling** in C to create a structured and efficient application. The design focuses on simplicity, clarity, and accuracy in billing operations, making it suitable for small and medium-sized restaurant setups.

Through this project, practical concepts of **structured programming, control flow, modular design, and data processing** have been implemented successfully. It not only enhances programming logic and problem-solving skills but also bridges the gap between academic learning and real-world application in the field of software development.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	

4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

1. INTRODUCTION

1.1 Objective

- To design and develop a **Hotel & Restaurant Billing System** using the C programming language.
- To automate the process of **order management, billing, and total calculation** for restaurant customers.
- To demonstrate **structured programming concepts** — arrays, loops, conditional statements, functions, and file handling (optional).

1.2 Overview

This system allows users (cashiers or waiters) to:

- View available menu items.
- Take multiple orders from customers.
- Calculate subtotal, tax, and grand total automatically.
- Generate a printable **bill receipt** with details like item name, quantity, and amount.

1.3 Features

- Interactive menu-driven interface.
- Menu stored in arrays for easy access.
- Calculates **total cost, GST (tax), and final bill**.
- Supports **multiple items per order**.
- Option to **save bill in a text file** (for record keeping).
- Displays **thank you receipt** after payment.

2. PROGRAM ANALYSIS

2.1 Problem Definition

A restaurant requires a billing program to quickly calculate customer bills, including:

- Displaying menu items with prices.
- Accepting multiple item orders.
- Calculating the total and tax automatically.
- Printing a clear bill to the customer.

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Loops, Functions, Arrays, Conditional Statements

Aspect	Description
I/O Type	Console-based
Optional	File Handling for saving receipts

2.3 Scopes and Limitations

Scope:

- Suitable for small and medium restaurants.
- Can easily be extended for inventory or database integration.

Limitations:

- Works only in text (console) mode.
- No graphical interface.
- Prices must be updated manually in source code.

3. SYSTEM DESIGN

3.1 Modules

The Hotel & Restaurant Billing System is divided into multiple functional modules, each handling a specific part of the program. These modules work together to simplify the billing process and make the system more efficient and user-friendly.

Module Name	Description
Menu Display Module	This module displays the complete list of food items and beverages available in the restaurant, along with their corresponding item numbers and prices. It allows the customer or cashier to view all options before placing an order.
Order Input Module	This module accepts the customer's choice of items and their quantities. It validates each input, ensuring that the selected item number exists and that the entered quantity is a valid number. It stores each order line for further processing.

Module Name	Description
Billing Calculation Module	This module computes the subtotal by multiplying the price and quantity for each ordered item. It then calculates the applicable GST (5%) and the final grand total. It ensures accuracy in the billing process by performing all mathematical computations automatically.
Bill Printing Module	This module generates a detailed bill format that includes the item name, quantity, price, subtotal, tax, and final total. It presents the bill in a neatly formatted tabular display for the customer.
File Storage Module (Optional)	This module stores each generated bill into a text file (bill.txt) for future record keeping. It appends new transactions to the file, acting as a simple sales record system for the restaurant.
Exit/Termination Module	This module safely terminates the program after completing all billing operations. It displays a closing message ("Thank you! Visit Again!") to indicate the end of the transaction.

Module Interaction Overview

1. Menu Display → shows available items.
2. Order Input → takes the customer's selection.
3. Billing Calculation → computes totals.
4. Bill Printing → displays the formatted receipt.
5. File Storage → saves bill data for records.
6. Exit Module → ends the program gracefully.

4. METHODOLOGY

The development of the *Hotel & Restaurant Billing System* project follows a structured methodology that includes five main stages: **Problem Analysis, System Design, Program Development, Testing & Validation, and Results & Documentation.**

4.1. Problem Analysis

In this stage, the existing manual restaurant billing process was studied and analyzed.

The main problems identified were manual calculation errors, time consumption, and lack of digital records. Based on this, clear objectives were defined — such as displaying a digital menu, taking multiple orders, calculating the total automatically, and generating a printed bill.

The problem was broken down into smaller tasks to make the program modular and efficient.

4.2. System Design

During the design phase, the logical structure of the system was created.

The main components identified were:

- Menu Display
- Order Entry
- Billing Calculation
- Bill Printing
- File Storage

Data structures like arrays and structures (struct MenuItem, struct Order) were designed to hold menu details and order information.

Flowcharts and algorithms were also prepared to visualize data flow and logical sequence before actual coding.

4.3. Program Development

This is the implementation phase where the design is translated into code using C programming language.

The program was written using modular programming concepts — dividing the system into small, manageable functions such as displaying the menu, taking inputs, computing totals, and printing the bill.

Control structures (loops, if-else, switch), arrays, and file handling were used to make the program interactive and functional.

4.4. Testing and Validation

After coding, each module was tested individually to check correctness.

Test cases were created for different scenarios — valid inputs, invalid item numbers, zero quantity, and multiple item orders.

Calculations for subtotal, GST, and total amount were verified manually to ensure accuracy.

The system passed all test cases successfully and was validated to perform error-free billing operations.

4.5. Result and Documentation

Finally, the program produced the desired result — a formatted and accurate **customer bill**. It displayed the total, GST, and grand total clearly, and also stored each transaction in a file for record maintenance.

Proper documentation was prepared, including **algorithm, flowchart, module descriptions,**

sample I/O screens, and conclusion, ensuring the project is easy to understand, maintain, and enhance in the future.

4.6. Algorithm

An **algorithm** is a **step-by-step logical procedure** used to solve a specific problem or perform a computation.

In programming, it defines the exact sequence of actions the computer must follow to achieve the desired output from a given set of inputs.

For the **Hotel & Restaurant Billing System**, the algorithm plays a key role in automating the billing process by defining how the program accepts orders, processes calculations, and generates the final bill.

Purpose of the Algorithm

The main purpose of using an algorithm in this project is to:

- Define the **logical flow** of the billing operations.
- Ensure **accuracy** in calculations like subtotal, GST, and total amount.
- Provide a **structured path** for program development before actual coding.
- Make the system easy to understand, debug, and enhance in the future.

Algorithm in the Project

The algorithm designed for the Hotel & Restaurant Billing System performs the following key tasks in order:

1. **Start the program** and display the system title.
2. **Display the menu** with available items and their prices.
3. **Accept user input** for the number of different items to order.
4. For each item:
    - Read the item number and quantity.
    - Validate the input to ensure correct data.
    - Calculate the **line total** (price × quantity).
5. **Compute subtotal**, **GST (5%)**, and **grand total**.
6. **Display the detailed bill** showing each ordered item, quantity, and amount.
7. **Optionally save the bill** to a file for record-keeping.
8. **Display a thank-you message** and end the program.

Importance of Algorithm in the Project

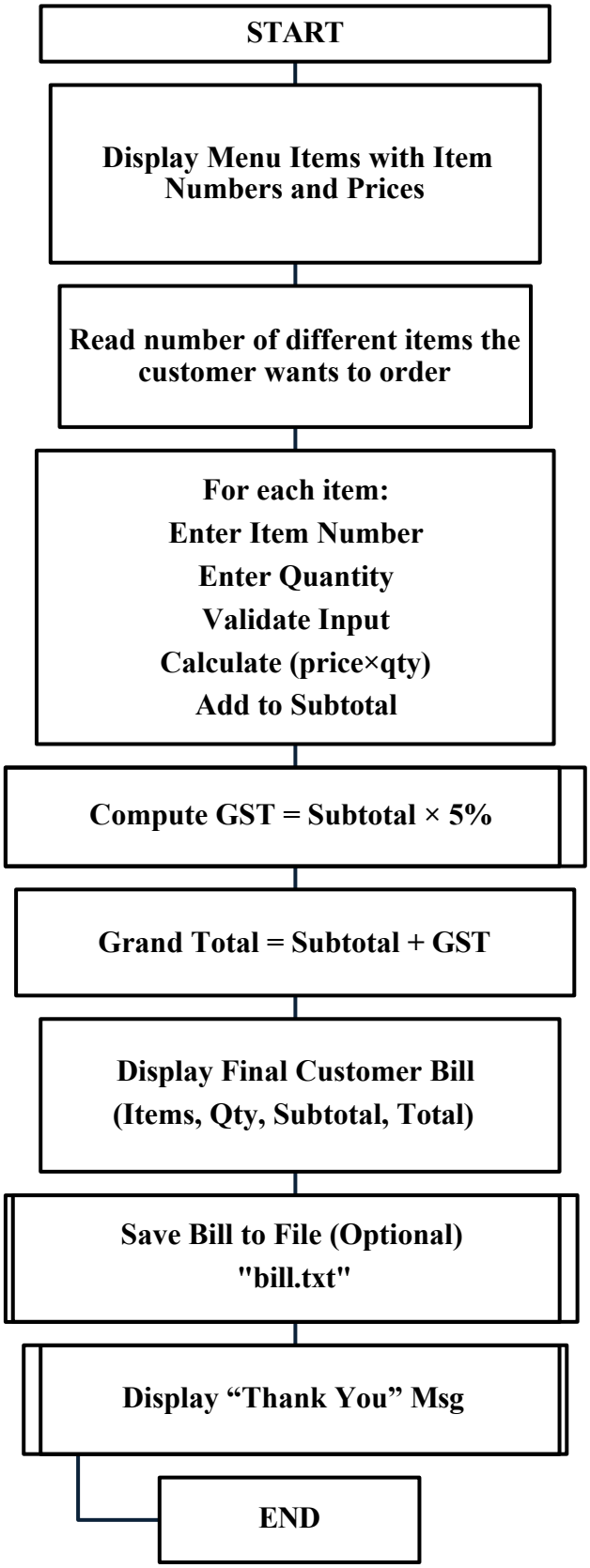
- Helps to convert **real-world billing logic into programmable steps**.
- Ensures **systematic flow of control** and logical correctness.
- Acts as a **blueprint for coding** and debugging.
- Provides **clarity, modularity, and efficiency** in the program design.

4.7. Flowchart

The flowchart represents the logical flow of the *Hotel & Restaurant Billing System*. It visually outlines the sequence of operations — from displaying the menu to generating the customer bill — ensuring clarity in how data moves through each stage of the program.

Explanation of Flow

1. **Start** – Program begins execution.
2. **Display Menu** – Lists all food items, their numbers, and prices.
3. **Take Order Count** – The user enters how many different items are being ordered.
4. **Process Each Order** –
    - Accepts item number and quantity.
    - Checks for validity.
    - Calculates amount for that item and updates subtotal.
5. **Calculate Tax and Total** – Computes GST (5%) and adds it to the subtotal.
6. **Display Bill** – Prints a formatted bill showing items, prices, subtotal, GST, and grand total.
7. **Save Bill** – Optionally stores the bill in a text file for record-keeping.
8. **End** – Displays a thank-you message and exits the program.



5. SOURCE CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>

#define MAX_ITEMS 20

struct MenuItem {
    int id;
    char name[30];
    float price;
};

struct Order
    { int
    itemId; int
    quantity;
    float total;
};

int main() {
    struct MenuItem menu[] = {
        {1, "Veg Fried Rice", 120.0},
        {2, "Chicken Fried Rice", 150.0},
        {3, "Paneer Butter Masala", 180.0},
        {4, "Chicken 65", 160.0},
        {5, "Veg Noodles", 110.0},
        {6, "Parotta (2pcs)", 40.0},
        {7, "Meals (Veg)", 100.0},
        {8, "Meals (Non-Veg)", 130.0},
        {9, "Ice Cream", 60.0},
        {10,"Fresh Lime Juice", 40.0}
    };
    int n = sizeof(menu)/sizeof(menu[0]);
    struct Order orderList[MAX_ITEMS];
    int orderCount = 0, choice, qty;
    float subtotal = 0.0, gst, grandTotal;

    printf("\n==============================\n");
    printf("  HOTEL & RESTAURANT BILLING SYSTEM");
    printf("\n==============================\n");
    printf("\nMenu:\n");
    printf("----------------------------\n");
    printf("No.  Item Name\t\t\tPrice (Rs)\n");
    printf("----------------------------\n");
    for(int i=0; i<n; i++)
```

```
    printf("%-3d  %-25s %.2f\n", menu[i].id, menu[i].name, menu[i].price);
printf("----------------------------\n");

printf("\nEnter number of different items ordered: ");
scanf("%d", &orderCount);

for(int i=0; i<orderCount; i++)
    { printf("\nEnter item number:
    "); scanf("%d", &choice);
    printf("Enter quantity: ");
    scanf("%d", &qty);

    if(choice >= 1 && choice <= n)
        { orderList[i].itemId = choice;
        orderList[i].quantity = qty;
        orderList[i].total = menu[choice-1].price * qty;
        subtotal += orderList[i].total;
    } else {
        printf("Invalid item number!\n");
        i--; // repeat iteration
    }
}

gst = subtotal * 0.05; // 5% GST
grandTotal = subtotal + gst;

// Print Bill
printf("\n\n============= CUSTOMER BILL =============\n");
printf("%-3s %-25s %-10s %-10s\n", "No.", "Item", "Qty", "Amount");
printf("----------------------------\n");

for(int i=0; i<orderCount; i++)
    { int id = orderList[i].itemId - 1;
    printf("%-3d %-25s %-10d %-10.2f\n",
            menu[id].id, menu[id].name,
            orderList[i].quantity,
            orderList[i].total);
}

printf("----------------------------\n");
printf("Subtotal\t\t\t%.2f\n", subtotal);
printf("GST (5%%)\t\t\t%.2f\n", gst);
printf("Grand Total\t\t\t%.2f\n", grandTotal);
```

```
    printf("----------------------------\n");
    printf("Thank you! Visit Again!\n");

    // Save to file (optional)
    FILE *fp = fopen("bill.txt", "a");
    if(fp != NULL) {
        fprintf(fp, "\n==== Customer Bill ====\n");
        for(int i=0; i<orderCount; i++) {
            int id = orderList[i].itemId - 1;
            fprintf(fp, "%s x %d = %.2f\n",
                    menu[id].name, orderList[i].quantity, orderList[i].total);
        }
        fprintf(fp, "Subtotal: %.2f\nGST(5%%): %.2f\nGrand Total: %.2f\n", subtotal, gst,
grandTotal);
        fprintf(fp, "----------------\n");
        fclose(fp);
    } else {
        printf("Error saving bill file!\n");
    }

    return 0;
}
```

SOURCE CODE EXPLANATION:

1) Headers &

Macros #include

<stdio.h> #include

<stdlib.h> #include

<string.h>

#define MAX_ITEMS 20

- stdio.h for input/output, stdlib.h for general utilities, string.h for string ops (safe to include even if minimal use).
- MAX_ITEMS limits how many different items can be added to one bill (safety cap).

---

2) Data Models (Structures)

struct MenuItem

{ int id;

```
    char name[30];
    float price;
};

struct Order
    { int
    itemId; int
    quantity;
    float total;
};
```

- **MenuItem** holds menu row: number, name, unit price.
- **Order** holds one line of the bill: which item (itemId), how many (quantity), and computed total (price × qty).

---

3) Main Function — Variables & Menu Setup

```
struct MenuItem menu[] = { ... };
int n = sizeof(menu)/sizeof(menu[0]);
struct Order orderList[MAX_ITEMS];
int orderCount = 0, choice, qty;
float subtotal = 0.0, gst, grandTotal;
```

- **Menu array**: predefined list of 10 items (id, name, price).
- **n**: number of menu items (auto-calculated).
- **orderList**: stores each selected item and quantity (up to MAX_ITEMS).
- **subtotal/gst/grandTotal** track bill math.

---

4) Print Title & Menu

```
printf("\n==============================\n");
printf("   HOTEL & RESTAURANT BILLING SYSTEM");
printf("\n==============================\n");
printf("\nMenu:\n");
```

...

```
for (int i=0; i<n; i++)
    printf("%-3d  %-25s %.2f\n", menu[i].id, menu[i].name, menu[i].price);
```

- Neatly shows system title and a formatted **price list**.
- %-25s left-aligns the item name column so it looks tabular.

---

5) Ask "How Many Different Items?"

```
printf("\nEnter number of different items ordered:
"); scanf("%d", &orderCount);
```

- orderCount = how many **distinct** items the customer wants (not total quantity). Example: 3 lines could be: "Fried Rice ×2", "Chicken 65 ×1", "Ice Cream ×3".

---

6) Loop to Capture Each Ordered Item

```
for (int i=0; i<orderCount; i++)
    { printf("\nEnter item number: ");
    scanf("%d", &choice);
    printf("Enter quantity: ");
    scanf("%d", &qty);

    if (choice >= 1 && choice <= n)
        { orderList[i].itemId = choice;
        orderList[i].quantity = qty;
        orderList[i].total = menu[choice-1].price * qty;
        subtotal += orderList[i].total;
    } else {
        printf("Invalid item number!\n");
        i--; // redo this iteration
    }
}
```

- For each bill line:
    1. Read **item number** (choice) and **quantity** (qty).
    2. Validate choice is within menu range.
    3. Compute line **total = price × qty** and add to **subtotal**.
    4. If invalid id, **retry** same index by i--.

---

7) Calculate Tax and Final Amount gst = subtotal * 0.05; // 5% GST grandTotal = subtotal + gst;

- Simple percentage calculation.
- You can parameterize 5% later if your taxes vary.

---

8) Print Customer Bill

```
printf("\n\n============= CUSTOMER BILL =============\n");
printf("%-3s %-25s %-10s %-10s\n", "No.", "Item", "Qty", "Amount");
printf("----------------------------\n");

for(int i=0; i<orderCount; i++)
   { int id = orderList[i].itemId - 1;
   printf("%-3d %-25s %-10d %-10.2f\n",
         menu[id].id, menu[id].name,
         orderList[i].quantity,
         orderList[i].total);
}

printf("----------------------------\n");
printf("Subtotal\t\t\t%.2f\n", subtotal);
printf("GST (5%%)\t\t\t%.2f\n", gst);
printf("Grand Total\t\t\t%.2f\n", grandTotal);
```

```
printf("----------------------------\n");
printf("Thank you! Visit Again!\n");
```

- Prints a **tabular bill** with:
  - Item number, name, qty, and line amount.
  - Subtotal, GST, and **Grand Total**.
- %10.2f shows amounts with **2 decimal places**.

---

9) Save Bill to File (Optional)

```
FILE *fp = fopen("bill.txt", "a");
if (fp != NULL) {
    fprintf(fp, "\n==== Customer Bill ====\n");
    for (int i=0; i<orderCount; i++) {
        int id = orderList[i].itemId - 1;
        fprintf(fp, "%s x %d = %.2f\n",
                menu[id].name, orderList[i].quantity, orderList[i].total);
    }
    fprintf(fp, "Subtotal: %.2f\nGST(5%%): %.2f\nGrand Total: %.2f\n",
            subtotal, gst, grandTotal);
    fprintf(fp, "----------------\n");
    fclose(fp);
} else {
    printf("Error saving bill file!\n");
}
```

- Opens **bill.txt** in **append** mode to keep a running ledger.
- Writes each line, then totals, then a separator.
- If file cannot be opened (permissions/path), informs the user.

---

10) Program Exit

```
return 0;
```

- Normal termination of main().

Control Flow Summary

1. Show title and menu list.
2. Ask how many **different** items.
3. For each item: read item number + quantity, compute line total, accumulate **subtotal**.
4. Compute **GST** and **Grand Total**.
5. Print a clean **receipt**.
6. Optionally **append** to bill.txt.
7. Exit.

6. INPUT AND OUTPUT SCREENS

Input Screen 1:

```
==============================
   HOTEL & RESTAURANT BILLING SYSTEM
==============================

Menu:
-------------------------------------------
No.   Item Name                 Price (Rs)
-------------------------------------------
1    Veg Fried Rice            120.00
2    Chicken Fried Rice        150.00
3    Paneer Butter Masala      180.00
4    Chicken 65                160.00
5    Veg Noodles               110.00
6    Parotta (2pcs)            40.00
7    Meals (Veg)               100.00
8    Meals (Non-Veg)           130.00
9    Ice Cream                 60.00
10   Fresh Lime Juice          40.00
-------------------------------------------

Enter number of different items ordered:
```

Input Screen 2:

```
Enter number of different items ordered: 3

Enter item number: 1
Enter quantity: 1

Enter item number: 4
Enter quantity: 1

Enter item number: 10
Enter quantity: 1
```

Output Screen:

```
============= CUSTOMER BILL =============
No. Item                        Qty        Amount
-------------------------------------------
1   Veg Fried Rice              1          120.00
4   Chicken 65                  1          160.00
10  Fresh Lime Juice            1          40.00
-------------------------------------------
Subtotal                          320.00
GST (5%)                          16.00
Grand Total                       336.00
-------------------------------------------
Thank you! Visit Again!

--------------------------------
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

This project successfully automates restaurant billing by:

- Using **structured programming** and modular design.
- Applying **loops, arrays, and structures** for data management.
- Demonstrating real-world utility with file saving (optional).

The system is efficient, user-friendly, and suitable for small restaurants.

7.2 Conclusion

The project meets its intended objectives:

- Simple yet functional **billing automation**.
- Demonstrated **real-life application of C programming** concep

- Easy to extend for further features like login system, discount, and database storage.

8. REFERENCES

- Kernighan & Ritchie, *The C Programming Language*, Prentice Hall, 1988.
- TutorialsPoint – C Programming
- GeeksforGeeks – Restaurant Billing in C
- Dev-C++ User Manual